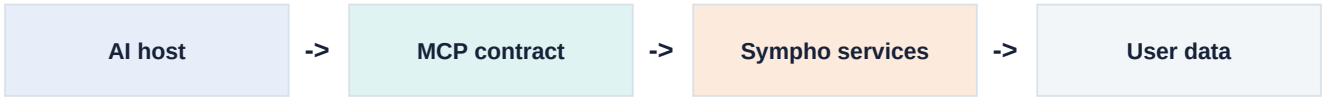
TECHNICAL DEEP DIVE

How AI agents connect to applications through MCP

A practical architecture, protocol, security, and implementation study for Sympho



Bottom line

MCP is not an AI brain and it does not reverse-engineer an application. It is a standard contract that lets an AI host discover named capabilities, submit validated structured arguments, receive results, and continue reasoning. For Sympho, the durable target is a remote MCP service over a shared backend, with SwiftData retained as the offline cache.

Prepared for Sympho

Research current as of 19 June 2026

Scope: MCP protocol mechanics, Codex, Claude Code, Cursor, hosted app patterns, automatic generation claims, Supabase integration, security, and a concrete Sympho rollout.

Executive summary

Recommended direction

Build a remote Sympho MCP server backed by a real authenticated application service and Supabase. Do not make external agents open or mutate the SwiftData store directly. Use a local MCP only as a short-lived prototype or when a capability truly requires access to the user's Mac.

An AI agent interacts with an application through an explicit capability loop:

Discover -> **Select** -> **Approve** -> **Execute** -> **Observe**

- The MCP server publishes tool names, descriptions, JSON input schemas, output schemas, annotations, resources, prompts, and optional server-level instructions.
- The host application - Codex, Claude Code, Cursor, ChatGPT, or another client - obtains these definitions through MCP and makes the relevant definitions available to its model.
- The model does not directly call Sympho. It proposes a structured tool call. The host applies policy and approval rules, then the MCP client sends a JSON-RPC request to the Sympho MCP server.
- The server authenticates the user, validates the arguments, invokes Sympho's domain/application service, records an audit event, and returns a structured result.
- The model sees the result and may answer, correct its plan, or call another tool.

What 'automatic MCP generation' actually means

Products such as Cursor can use an agent to scaffold an MCP server from specifications, SDK documentation, an API schema, or an existing codebase. Cursor also supports one-click installation links and curated server listings. These features automate code generation or configuration. They do not remove the need to define trustworthy app operations, authentication, permissions, validation, error semantics, and production deployment. Cursor's own guide describes using the specification and SDK documentation to scaffold a PostgreSQL MCP server; it still produces real server code that must be run and maintained. [8][9]

What this means for Sympho

Today	Required bridge	Target
Local SwiftData models; view-level mutations; simulated Supabase sync; in-app OpenAI draft generation.	Shared application service, PostgreSQL schema, auth/RLS, real sync, typed MCP tools, audit and concurrency controls.	Agents work while Sympho is closed; changes appear in every signed-in client after sync; user permissions remain authoritative.

Contents

Executive summary 2

What 'automatic MCP generation' actually means . . . 2

What this means for Sympho . . . 2

Contents 3

1. The correct mental model 6

The five actors . . . 6

What MCP is not . . . 6

2. Protocol anatomy 7

Initialization and capability negotiation . . . 7

The three core server primitives . . . 7

Tool discovery . . . 7

Tool execution . . . 7

3. How the AI knows what to do 8

End-to-end decision loop . . . 8

Concrete Sympho example . . . 8

Why names and descriptions matter . . . 8

4. Local, remote, and hybrid MCP 10

Sympho local prototype . . . 10

Sympho production target . . . 10

Hybrid pattern . . . 10

5. What 'automatic MCP' really means 11

Cursor specifically . . . 11

Three levels of generated quality . . . 11

6. How major AI clients consume MCP 12

Codex . . . 12

Claude Code . . . 12

Cursor . . . 12

ChatGPT and API-hosted agents . . . 12

7. Notion as the relevant product pattern **13**

What Notion actually built . . . 13

What Sympho should copy . . . 13

8. Sympho: current integration readiness **14**

What exists . . . 14

Important domain ambiguities to resolve . . . 14

Why these questions matter to MCP . . . 14

9. Recommended Sympho architecture **15**

Target topology . . . 15

Application service shape . . . 15

Hosting choice . . . 15

10. Proposed Sympho tool contract **16**

Phase 1: read tools . . . 16

Phase 2: controlled write tools . . . 16

Phase 3: workflow tools . . . 16

Annotations and approvals . . . 16

11. Identity, permissions, and synchronization **17**

OAuth flow for Sympho . . . 17

Suggested scopes . . . 17

Database ownership . . . 17

Sync model . . . 17

12. Security and safety model **19**

Primary threats . . . 19

Server-side non-negotiables . . . 19

13. Implementation roadmap **20**

Stage 0 - product contract . . . 20

Stage 1 - application service and tests . . . 20

Stage 2 - cloud source of truth . . . 20

Stage 3 - read-only MCP beta . . . 20

Stage 4 - controlled writes . . . 20

Stage 5 - workflows and distribution . . . 20

Indicative engineering effort . . . 20

14. Testing and operational readiness **22**

Protocol verification . . . 22

Test pyramid . . . 22

Tool quality metrics . . . 22

Versioning . . . 22

15. Decisions for Sympho **23**

Recommended decisions . . . 23

First vertical slice . . . 23

Questions to answer before implementation . . . 23

Appendix A. Example protocol exchange **24**

1. Discovery . . . 24

2. Model-selected call . . . 24

3. Structured result . . . 24

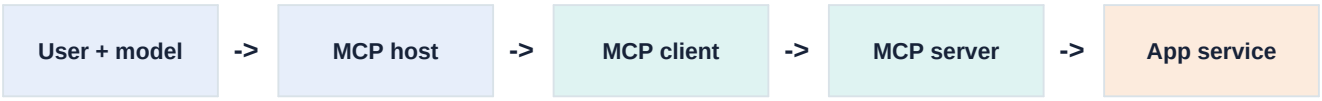
4. Correctable conflict . . . 24

Appendix B. Sympho repository evidence **25**

References **26**

1. The correct mental model

MCP is a protocol between an AI application and capability providers. The official architecture separates the host, a client connection per server, and focused MCP servers. The protocol standardizes context exchange; it does not specify how a model reasons, how an app stores data, or how a company designs permissions. [1][2]



The five actors

Actor	Responsibility	Sympho example
User	States intent and approves consequential actions.	Create a Swedish C1 learning plan.
Model	Chooses a plan and proposes tool calls from the definitions it receives.	Selects create_domain, then create_track, module, and node tools.
Host	Runs the conversation, model, policy, approval UI, and MCP clients.	Codex, Claude Code, Cursor, or ChatGPT.
MCP client	Maintains one protocol connection and sends discovery/call messages.	The Sympho connection inside Codex.
MCP server	Publishes capabilities and translates calls into application operations.	api.sympho.app/mcp.
Application service	Owns validation, transactions, authorization, and domain invariants.	SymphoCommandService backed by Postgres.

What MCP is not

- It is not a database driver that should be given unrestricted production SQL access.
- It is not a screen automation protocol. Computer-use tools can click an app, but MCP normally exposes semantic operations below the UI.
- It is not a substitute for an API or application service. A server still needs a safe way to read and change application state.
- It is not automatic understanding of every installed app. The client must be configured, installed through a trusted flow, or discover a known server through a registry/gallery.
- It is not an authorization system by itself. Remote deployments usually pair MCP with OAuth and server-side access control.

> **A useful analogy**
>
> An MCP server is closer to a strongly described remote control than to an autonomous employee. The model decides which button to request; the host and server decide whether that button may actually be pressed.

2. Protocol anatomy

Initialization and capability negotiation

An MCP session begins with initialization. Client and server negotiate protocol versions and capabilities. A server may advertise support for tools, resources, prompts, list-change notifications, logging, or other protocol features. A host can therefore adapt without hard-coding every server. [1][3]

```
Client -> Server: initialize
Server -> Client: protocolVersion, capabilities, serverInfo, instructions
Client -> Server: notifications/initialized
```

The three core server primitives

Primitive	Control	Purpose	Sympho example
Tools	Model-controlled request	Execute an operation or retrieve information.	create_node, search_workspace
Resources	Application-controlled context	Expose passive, addressable content.	sympho://domains/{id}/outline
Prompts	User-selected template	Provide reusable workflows or examples.	Plan a learning track

The MCP specification describes prompts as user-controlled, resources as application-controlled, and tools as model-controlled. 'Model-controlled' means the model may request a tool; it does not mean the server must execute without policy checks or user consent. [2][3]

Tool discovery

The client calls `tools/list`. The server returns a list of tool definitions. Each definition can include a stable name, title, detailed description, JSON input schema, output schema, annotations, and extension metadata. The client may refresh the list when the server emits a list-changed notification. [2][3]

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/list"
}

{
  "name": "create_node",
  "description": "Create one learning node in a known module.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "module_id": {"type": "string", "format": "uuid"},
      "title": {"type": "string", "minLength": 1},
      "description": {"type": "string"},
      "idempotency_key": {"type": "string"}
    },
    "required": ["module_id", "title", "idempotency_key"]
  }
}
```

Tool execution

To execute, the client sends `tools/call` with the selected tool name and arguments. The server validates both the protocol-level schema and business rules, performs the operation, then returns text and/or structured content. An application-level failure should be represented clearly enough for the model to correct its plan. [2]

3. How the AI knows what to do

The model's apparent understanding comes from a combination of tool definitions, server instructions, current conversation context, tool results, and model training. No single field is sufficient. Good integration quality is largely interface design quality.

End-to-end decision loop

#	Stage	What happens
1	Connect	The host opens a stdio or HTTP session.
2	Discover	The client gets capabilities and available tool schemas.
3	Contextualize	The host gives relevant tool definitions and instructions to the model.
4	Reason	The model maps user intent to a tool and forms JSON arguments.
5	Authorize	The host checks tool policy and may request user approval.
6	Execute	The MCP server validates identity, permissions, schema, and domain rules.
7	Return	The server returns a structured result, error, or progress.
8	Continue	The model observes the result and answers or invokes another tool.

Concrete Sympho example

User request: *'Build a six-week Swift concurrency learning track and save it under Programming.'*

Model action	Why	Expected result
search_workspace	Resolve the Programming domain and avoid duplicates.	Domain ID and existing related tracks.
create_track	Create the parent structure.	Track ID, version, timestamps.
create_module_batch	Create an ordered curriculum atomically.	Module IDs in final order.
create_node_batch	Populate focused learning units.	Node IDs and validation summary.
get_track_outline	Verify what was actually committed.	Canonical track snapshot.

Why names and descriptions matter

- A vague tool named `manage` forces the model to guess. Prefer semantic verbs such as `create_node` or `move_node`.
- Descriptions should state preconditions, side effects, when not to use the tool, and any important ordering constraints.
- Schemas should use enums, UUID formats, minimum lengths, bounded arrays, and meaningful field descriptions.
- Results should return stable IDs and canonical state, not only 'success'. The model needs evidence to continue correctly.
- Server instructions can explain cross-tool workflows and global constraints. Codex currently reads MCP server instructions and recommends putting essential guidance early. [11]

Design consequence

An MCP server can be protocol-correct and still perform badly if its tools are ambiguous. Tool evaluation should test whether models select the right operation, provide valid arguments, recover from errors, and stop when the task is complete.

SECTION TAKEAWAY

03

Interface quality is agent quality

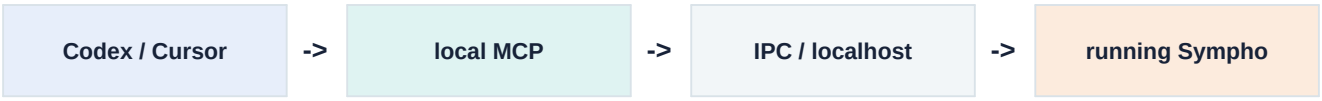
SELECTION	Does the model choose the right tool?
ARGUMENTS	Does it produce valid, bounded input?
RECOVERY	Can it correct errors without unsafe retries?

4. Local, remote, and hybrid MCP

MCP defines local stdio and remote Streamable HTTP as the principal transport patterns. The protocol is the same conceptually, but deployment, availability, authentication, and risk differ. [1][4]

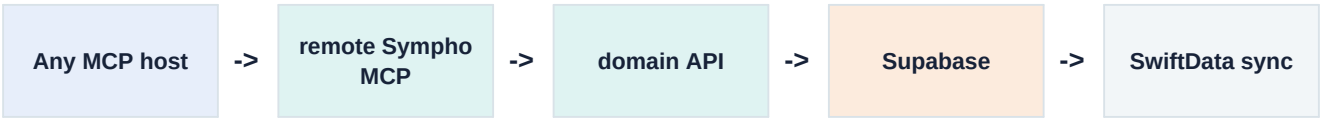
Dimension	Local stdio	Remote Streamable HTTP
Lifecycle	Host launches a child process.	Independent service stays online.
Reach	Same machine and user session.	Any authorized client with network access.
Auth	Usually environment/config and OS boundary.	OAuth 2.1 or bearer credentials plus app authorization.
App availability	Local process or desktop app often must be available.	Desktop/mobile app may be closed.
Best fit	Filesystem, desktop automation, hardware, prototypes.	SaaS data, multi-device apps, team integrations.
Primary risk	Executing untrusted local code with user privileges.	Token, session, tenant isolation, prompt-injection and service exposure.

Sympho local prototype



A local prototype can prove tool design quickly. The MCP process should talk to a controlled app service through IPC or localhost. It should not open the SwiftData store as an undocumented shared database. The current macOS target has a network-client entitlement but no network-server entitlement, so an embedded localhost HTTP server would require entitlement and sandbox work.

Sympho production target



With a remote service, an agent can work while the native app is closed. Supabase stores the authoritative data, the MCP service performs user-scoped operations, and native clients synchronize changes when online. The app remains local-first, but not local-only.

Hybrid pattern

A hybrid deployment may expose cloud-backed knowledge and planning tools remotely while keeping truly local operations - for example importing a file from a user-selected folder - in a signed local companion. Avoid duplicating the same write operation in local and remote implementations unless both call the same application contract.

> **Recommendation for Sympho**
>
> Use remote Streamable HTTP for the core workspace. Add a local companion only if a later workflow genuinely needs local files or desktop control. This keeps the central integration available, multi-client, and compatible with Notion-style OAuth onboarding.

5. What 'automatic MCP' really means

The word 'automatic' is used for several different product behaviors. They should not be conflated.

Claim	What is automated	What remains your responsibility
AI-generated MCP server	Agent writes scaffold and tool handlers from docs/code/API schemas.	Correct semantics, auth, permissions, testing, deployment, operations.
One-click install	Client writes config or registers a server URL/command.	Trusting the provider and approving local execution/OAuth access.
Bundled plugin	Plugin installs skills, prompts, and an MCP server together.	Provider still implemented and maintains the server.
Curated directory	Client displays known providers and install actions.	No automatic understanding of unlisted apps.
Dynamic discovery	Connected server changes its own advertised tools at runtime.	Server must generate accurate schemas and enforce access.
API wrapping	Generator turns OpenAPI/SDK operations into candidate tools.	Tool curation; raw APIs are rarely ideal model interfaces.

Cursor specifically

Cursor supports stdio, SSE, and Streamable HTTP servers, can use configured MCP tools in Agent, provides an MCP directory, and lets providers create an 'Add to Cursor' installation link. Its official tutorial shows Cursor scaffolding a PostgreSQL server after being given the MCP specification, SDK documentation, and database library documentation. [8][9]

Therefore, **Cursor can build the adapter code**, but it cannot infer a safe product contract from nothing. If asked to wrap the current Sympho repository directly, it would still need decisions such as whether a node may belong to both a module and project, how conflicts are resolved, what deletion means, which fields are user-editable, and what data each OAuth user may access.

Three levels of generated quality

Level	Typical result	Use
Protocol scaffold	Server starts, tools/list works, sample calls execute.	Prototype only.
Application adapter	Tools call stable services with validation and tests.	Internal beta.
Product integration	OAuth, tenant isolation, consent, audit, metrics, retries, versioning, documentation.	External users and production.

Practical interpretation

Use Codex, Claude Code, or Cursor to accelerate implementation. Do not use generated MCP code as evidence that the application's authorization and domain boundaries are correct.

6. How major AI clients consume MCP

Codex

Current Codex documentation supports local stdio and remote Streamable HTTP MCP servers in the CLI and IDE extension. Configuration is shared through `config.toml`; remote servers can use bearer tokens or OAuth, and tool allow/deny lists plus approval modes can be configured. Codex also consumes the server's initialization instructions. [11]

```
[mcp_servers.sympho]
url = "https://api.sympho.app/mcp"
enabled_tools = ["search_workspace", "get_item", "create_node"]
default_tools_approval_mode = "prompt"

# After configuration:
# codex mcp login sympho
```

Claude Code

Claude Code supports local stdio and remote HTTP MCP servers, user/project/local configuration scopes, and browser-based OAuth for remote servers. Project-scoped configuration can be shared in `.mcp.json`, with an explicit trust step. [10]

```
claude mcp add --transport http sympho https://api.sympho.app/mcp
# Then use /mcp to authenticate and inspect the connection.
```

Cursor

Cursor supports MCP tools, resources, prompts, roots, and elicitation across local and remote transports. It can auto-run configured tools if the user enables that behavior. The presence of auto-run makes accurate side-effect descriptions and server-side authorization especially important. [8]

```
{
  "mcpServers": {
    "sympho": {
      "url": "https://api.sympho.app/mcp"
    }
  }
}
```

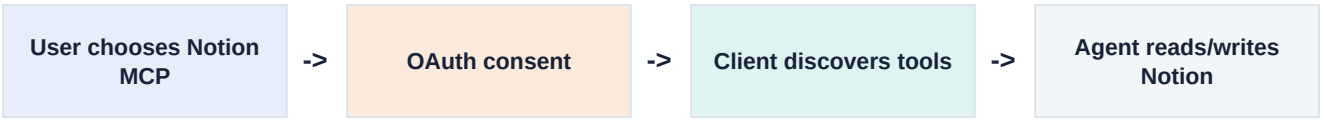
ChatGPT and API-hosted agents

OpenAI supports remote MCP integrations for ChatGPT and the Responses API. Remote deployments are required for cloud-hosted clients because they cannot launch a local process on the user's Mac. OpenAI recommends OAuth for private remote data and emphasizes prompt-injection and data-exfiltration risks. [16]

> **Compatibility rule**
>
> Design the Sympho MCP service against the official protocol and conservative common features. Treat client-specific configuration, approval UI, and tool naming constraints as adapters around the same server contract.

7. Notion as the relevant product pattern

Notion's current recommended integration is a hosted MCP endpoint at `https://mcp.notion.com/mcp`. Users connect from Codex, Claude Code, Cursor, ChatGPT, and other clients, then complete an OAuth flow. The hosted server is maintained by Notion and exposes tools optimized for agents. [12]



What Notion actually built

- A continuously available remote MCP service.
- OAuth connection to a specific user's workspace and existing permissions.
- Agent-oriented tools such as search and fetch rather than exposing raw internal storage.
- Cross-client setup instructions and stable URLs.
- Rate limits, security guidance, and a maintained product lifecycle.

What Sympho should copy

Notion pattern	Sympho equivalent
Hosted endpoint	https://api.sympho.app/mcp
OAuth workspace consent	Sign in to Sympho and approve client/scopes.
Existing user permission model	RLS on every workspace-owned table and storage object.
Agent-optimized tools	Create learning structures, search knowledge, attach resources, update progress.
App need not be running	Postgres is authoritative; SwiftData synchronizes later.

The key lesson is not 'Notion has an MCP server.' It is that Notion already had a network-accessible product API, authentication system, permission model, and canonical cloud data. MCP is the agent-friendly facade over that foundation. Sympho currently has the native model but not yet the equivalent cloud application boundary.

8. Sympho: current integration readiness

The repository inspection shows a strong local-first model and an early AI generation abstraction, but no external application interface.

What exists

Area	Evidence	Assessment
Data model	14 SwiftData model types registered in SymphoApp.swift.	Useful domain vocabulary and UUID identities.
Hierarchy	Domain, Track, Module, Project, Node, Resource plus reading/planner/library models.	Good candidates for typed MCP operations.
Local persistence	Persistent ModelContainer in the native app.	Fast offline UX, but inaccessible to remote agents.
Sync	hasConfiguredRemoteBackend = false; push/pull bodies are placeholders.	No authoritative remote state today.
AI	AIService generates drafts through provider adapters.	Opposite direction: Sympho calls AI; it does not expose Sympho to AI.
Mutations	ModelContext inserts/saves are spread through many SwiftUI views.	No reusable command boundary for MCP.
Sandbox	Network client entitlement only.	Embedded local HTTP would require additional server capability.

Important domain ambiguities to resolve

- Can a Module have both a direct Domain and a Track, or must exactly one parent path be set?
- Can a Node belong to a Module and a Project simultaneously? If so, which relationship defines curriculum ordering?
- Are soft deletes permanent tombstones, recoverable archives, or sync implementation details?
- Which timestamps are server-owned, and how are conflicting offline edits merged?
- Which models are private per user, shared within a workspace, or globally seeded?
- How are attachment bytes, markdown files, storage paths, and signed download URLs represented across devices?

Why these questions matter to MCP

A native UI can silently encode rules in screen structure and button availability. An external agent bypasses that visual guardrail. Those implicit rules must become explicit service validation, database constraints, tool descriptions, and test cases before write tools are safe.

> **Readiness conclusion**
>
> Sympho can support MCP, but the first engineering milestone is not the MCP endpoint. It is a shared, testable application service and remote source of truth that both the app and MCP server can use.

9. Recommended Sympho architecture

Target topology

Codex / Claude / Cursor -> **Sympho MCP** -> **Application service** -> **Supabase Postgres** -> **Native sync**

Layer	Responsibilities	Must not do
MCP protocol	Initialize, list/call tools, resources, results, annotations, transport.	Contain core domain rules.
Auth gateway	OAuth discovery, token validation, scopes, user identity, rate limits.	Trust client-supplied user IDs.
Application service	Commands, queries, invariants, transactions, idempotency, audit.	Depend on a specific AI model.
Persistence	RLS, constraints, relations, versions, tombstones, storage metadata.	Expose service-role credentials to clients.
Native sync	Outbox, pull cursor, conflict handling, attachment transfer, local cache.	Assume the native app is the only writer.

Application service shape

```
protocol SymphoWorkspaceService {
  func search(_ query: SearchQuery, actor: Actor) async throws -> SearchPage
  func createNode(_ command: CreateNode, actor: Actor) async throws -> NodeDTO
  func moveNode(_ command: MoveNode, actor: Actor) async throws -> NodeDTO
  func archiveItem(_ command: ArchiveItem, actor: Actor) async throws -> AuditResult
}

// UI, sync, REST, and MCP adapters call the same semantic operations.
```

This contract need not literally be a Swift protocol shared with the TypeScript server. The important point is one semantic command/query model with contract tests. It may be implemented through PostgreSQL functions, a private service API, or generated transport DTOs.

Hosting choice

Supabase now documents an OAuth 2.1 server for MCP authentication, including PKCE, dynamic client registration, and RLS-compatible user tokens; the feature is currently described as public beta. Supabase also documents running MCP servers on Edge Functions. Because auth and Edge runtime support are evolving, isolate the MCP service from the domain layer so hosting can move between a portable TypeScript service and Edge Functions without changing the tool contract. [13][14][15]

> **Pragmatic deployment**
>
> Start the production-quality MCP adapter as a small portable TypeScript service using the official MCP SDK. Use Supabase for Postgres, Storage, Auth, and RLS. Re-evaluate Edge Functions after end-to-end OAuth interoperability is verified with each target client.

10. Proposed Sympho tool contract

Start narrow. A small, composable, strongly typed tool set is easier for models to select correctly and easier for users to trust. Add workflow tools only after atomic primitives are stable.

Phase 1: read tools

Tool	Purpose	Key output
search_workspace	Search title, description, notes, tags, and resource bodies by allowed entity types.	Ranked compact results with IDs and paths.
get_workspace_outline	Return bounded hierarchy for one workspace/domain/track.	Nested summaries, cursors, versions.
get_item	Fetch canonical detail for one known ID.	Typed entity, parents, links, version.
list_recent_items	Review recent or changed content.	Cursor-paginated summaries.

Phase 2: controlled write tools

Tool	Important inputs	Safety semantics
create_domain	title, description, icon, color, idempotency_key	Additive; return canonical ID.
create_track	domain_id, title, description, position	Validate parent ownership.
create_module	track_id or domain_id, title, position	Enforce exactly one parent mode.
create_node	module_id/project_id, title, status, priority	Validate placement and enums.
create_project	domain_id/track_id, title, outcome	Additive.
create_resource	title, body or URL, destination IDs	URL validation; no arbitrary server file paths.
update_item	item_type, ID, patch, expected_version	Optimistic concurrency; bounded fields.
move_item	ID, target parent, expected_version	Transactional relationship update.
archive_item	ID, expected_version, reason	Destructive/confirm; reversible where possible.

Phase 3: workflow tools

- `create_learning_plan`: validate and commit a full domain/track/module/node graph atomically after returning a preview.
- `organize_inbox`: propose classifications first; apply only confirmed changes.
- `schedule_study_sessions`: produce a preview with timezone and conflicts before creating planner events.
- `import_resource`: use an upload-session or signed URL flow for binary attachments.

Annotations and approvals

The current schema defines hints such as `readOnlyHint`, `destructiveHint`, `idempotentHint`, and `openWorldHint`. These help clients present and gate tools, but the specification explicitly treats them as untrusted hints. Authorization must remain server-side. [3]

11. Identity, permissions, and synchronization

OAuth flow for Sympho

- The MCP endpoint returns an authorization challenge and protected-resource metadata location when a token is missing.
- The client discovers the authorization server and begins Authorization Code with PKCE.
- Sympho presents the requesting client, redirect URI, and requested scopes to the signed-in user.
- The authorization server issues access and refresh tokens bound to the MCP resource/audience.
- The MCP service validates issuer, audience, expiry, signature, and scopes on every request, then derives the user identity from the token.

Suggested scopes

Scope	Allows	Default
workspace:read	Search, outline, item detail.	Yes
workspace:write	Create and update non-destructive workspace content.	Optional consent
workspace:organize	Move/reorder items and relationship changes.	Step-up
workspace:archive	Archive or restore items.	Step-up
attachments:write	Create upload sessions and attach completed uploads.	Step-up
planner:write	Create or update planner events.	Step-up

Database ownership

Every user- or workspace-owned table needs an ownership key and RLS policy. Never accept `user_id` from a tool argument as authorization evidence. Derive it from the validated access token. Supabase recommends RLS for all exposed tables and warns that table grants and RLS are separate controls.

Sync model

Concern	Recommended rule
Authority	Postgres is canonical after remote integration; SwiftData is the local materialized cache.
Local writes	Write locally and enqueue an outbox operation with stable operation ID.
Remote writes	MCP writes commit server-side and increment version / updated_at.
Pull	Use a server cursor/change log, not only client wall-clock timestamps.
Conflict	Reject stale expected_version or apply a documented field-level merge policy.
Delete	Use archived_at/deleted_at tombstones retained long enough for every client to observe.
Attachments	Sync metadata separately from bytes; use checksums and signed storage operations.

Critical requirement

The existing SyncManager scans only a subset of models and uses a timestamp-shaped placeholder pull. External writers make real change tracking, deletion propagation, and conflict handling mandatory rather than optional.

SECTION TAKEAWAY

11

A second writer changes the sync problem

CURSOR	Pull from a durable change cursor, not wall-clock time.
VERSION	Reject stale writes with expected_version.
AUDIT	Record actor, client, operation, target, and result.

12. Security and safety model

MCP expands the application attack surface because model-selected actions combine user intent, untrusted content, external services, and privileged application operations. Security must assume that models can make mistakes and content can be adversarial. [5][16]

Primary threats

Threat	Sympho scenario	Mitigation
Prompt injection	A saved resource contains text telling the agent to export other private notes.	Treat resource text as data; isolate untrusted content; confirmation for cross-boundary writes/exports.
Over-broad tools	update_item can alter ownership or hidden fields.	Allow-listed patch schema; server-owned fields; per-operation authorization.
Confused deputy	MCP accepts a token intended for another API.	Audience-bound access tokens; never pass client token through to unrelated APIs.
Tenant escape	Client supplies another user's workspace_id.	Derive actor from token; RLS; object-level checks; negative tests.
Replay/duplicate	Agent retries create_learning_plan after timeout.	Idempotency key and stored operation result.
Stale overwrite	Agent updates a node after offline user edits it.	expected_version conflict response and refetch.
Local server compromise	One-click config launches a malicious package.	Signed distribution, visible command, sandbox, stdio, minimal privileges.
Data exfiltration	One MCP reads Sympho and another writes to an attacker-controlled service.	Least privilege, approval policy, outbound restrictions, user-visible destinations.

Server-side non-negotiables

- Validate token signature, issuer, audience, expiry, scope, session/revocation posture, and user status.
- Use RLS as defense in depth, but also authorize semantic commands in the application service.
- Never expose Supabase service-role or database credentials to an MCP client or native public client.
- Apply per-user and per-tool rate limits; bound result size and pagination.
- Log actor, client ID, tool, normalized input hash, affected IDs, result, latency, request ID, and approval context where available.
- Separate destructive tools, use reversible archive semantics first, and make bulk operations previewable.
- Avoid returning secrets, internal stack traces, storage paths, or unrelated object fields in errors.

> **Annotations are not enforcement**
>
> A tool marked read-only can still be implemented maliciously or incorrectly. Clients may use annotations for UX, but Sympho must enforce all permissions and side-effect limits regardless of what the model or client believes.

13. Implementation roadmap

Stage 0 - product contract

- Resolve hierarchy invariants, ownership model, deletion semantics, sharing model, and attachment lifecycle.
- Select the first three user journeys and define measurable success: search, create node, create learning structure.
- Write command/query DTOs independent of SwiftData and MCP.

Stage 1 - application service and tests

- Move mutations out of SwiftUI views into shared commands.
- Return typed errors: not_found, forbidden, conflict, validation_failed, rate_limited, internal.
- Add unit and contract tests for every invariant and side effect.

Stage 2 - cloud source of truth

- Create PostgreSQL schema, foreign keys, indexes, ownership columns, versions, tombstones, and audit tables.
- Enable RLS on every exposed table and storage path; test positive and negative tenant cases.
- Implement actual SwiftData outbox/pull/conflict synchronization across all supported models.

Stage 3 - read-only MCP beta

- Implement initialize, search_workspace, get_item, and get_workspace_outline.
- Use compact structured results, pagination, tool instructions, and per-user rate limits.
- Test in MCP Inspector, Codex, Claude Code, and Cursor.

Stage 4 - controlled writes

- Add create_node, create_module, create_track, and update_item with idempotency and expected_version.
- Enable OAuth consent and narrow scopes; mark side effects accurately; keep host approvals enabled.
- Run model-based tool-selection evaluations and adversarial prompt-injection tests.

Stage 5 - workflows and distribution

- Add preview/apply workflow tools and attachment upload sessions.
- Publish cross-client setup pages and verified one-click install links where supported.
- Add operational dashboards, error budgets, incident procedures, schema/tool version policy, and user revocation UI.

Indicative engineering effort

Workstream	Indicative scope	Main uncertainty
Domain service refactor	Several focused engineering days	Number of implicit UI rules.
Postgres/Auth/RLS	One to two engineering weeks	Sharing model and current Supabase beta behavior.
Production sync	One to three engineering weeks	Conflict and attachment requirements.
Read MCP vertical slice	Several days after API exists	Result design and client differences.
Write hardening	One to two weeks	Evaluation depth and approval UX.

These are architecture-scale estimates, not commitments. The dominant work is making Sympho a secure multi-writer application; protocol plumbing is the smaller component.

SECTION TAKEAWAY

13

Most of the work is product architecture

FIRST	Create a shared command and query boundary.
THEN	Build identity, RLS, remote truth, and real sync.
FINALLY	Expose a narrow MCP surface and evaluate it.

14. Testing and operational readiness

Protocol verification

Use the MCP Inspector to inspect initialization, tools, resources, prompts, notifications, invalid inputs, concurrent calls, and error responses before testing inside an AI client. The Inspector is transport-agnostic and should be the first debugging stop. [6]

Test pyramid

Layer	Examples	Pass condition
Schema	Missing required fields, enum errors, oversized arrays.	Rejected without side effects.
Domain	Invalid parent, stale version, duplicate idempotency key.	Deterministic typed result.
Authorization	Wrong user/workspace, missing scope, revoked session.	No data disclosure; correct 401/403 behavior.
Protocol	Initialize, list, call, pagination, cancellation, timeout.	Spec-compliant messages and bounded output.
Model selection	Natural-language prompts across multiple clients/models.	Correct tool and arguments above target rate.
Adversarial	Injected resource instructions, cross-MCP exfiltration attempts.	No unapproved sensitive read/write flow.
Sync	Offline native edit races with MCP edit/delete.	Documented merge or conflict; no silent loss.

Tool quality metrics

- Tool-selection accuracy and unnecessary-tool-call rate.
- Argument validation success on first attempt.
- Task completion rate and postcondition verification rate.
- Duplicate side-effect rate under retries/timeouts.
- Approval acceptance/denial rate by tool.
- P50/P95 latency, error rate, rate-limit rate, and result token size.
- Conflict rate and manual recovery rate.

Versioning

Treat tool names and schemas as public API. Prefer additive optional fields and new tool versions over silently changing behavior. Keep old tools available through a deprecation window, return clear replacement guidance, and evaluate both old and new contracts before migration.

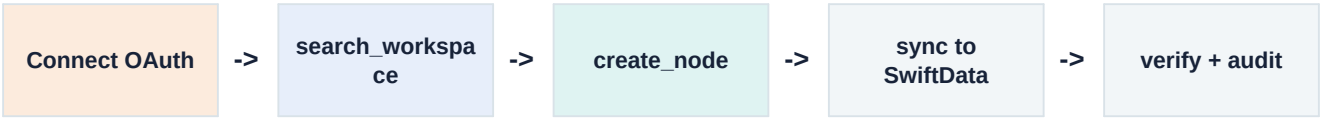
> **Definition of done for the first beta**
>
> A user can connect through OAuth, search only their own workspace, create one node exactly once under retries, see it appear in the native app after synchronization, inspect the audit event, and revoke the connection.

15. Decisions for Sympho

Recommended decisions

Decision	Recommendation	Reason
Primary deployment	Remote Streamable HTTP.	Works while app is closed and across clients/devices.
Source of truth	Supabase Postgres.	Matches existing architecture intent and enables shared access.
Native persistence	Keep SwiftData as offline cache.	Preserves immediate local UX.
MCP implementation	Portable TypeScript service first.	Mature SDK path and flexible hosting while auth features evolve.
Initial surface	Read tools plus create_node.	Smallest end-to-end proof with useful value.
Write policy	Approval by default; preview for bulk/destructive actions.	Limits model and injection mistakes.
Local server	Optional prototype/companion only.	Avoid two authoritative implementations.
Existing in-app AI	Deprioritize or make it another consumer of the service.	External agents provide reasoning; shared operations prevent duplication.

First vertical slice



This slice proves identity, permissions, semantic tool selection, a safe write, remote persistence, native synchronization, and observability. It also exposes the hardest architectural gaps early without committing to a large tool catalog.

Questions to answer before implementation

- Is Sympho single-user for the next release, or should the schema support collaborative workspaces immediately?
- Which entities must be available in the first MCP beta: learning hierarchy only, or planner, reading list, library, and dev captures too?
- Should bulk-generated learning plans be committed immediately, or always created as reviewable drafts?
- What is the expected offline conflict policy: last-write-wins, field merge, or explicit user resolution?
- Will ChatGPT/cloud clients be a launch requirement, or are coding clients sufficient for the first milestone?

Final conclusion

MCP is the correct integration mechanism for making Sympho usable by Codex, Claude Code, Cursor, ChatGPT, and future agent hosts. The decisive engineering work is to turn Sympho's current local UI-driven data mutations into a secure, authenticated, multi-writer application service. Once that boundary exists, the MCP adapter becomes focused, testable, and replaceable.

Appendix A. Example protocol exchange

The following is illustrative JSON-RPC, simplified for readability.

1. Discovery

```
-> {"jsonrpc":"2.0","id":1,"method":"initialize","params":{
     "protocolVersion":"2025-11-25",
     "clientInfo":{"name":"codex","version":"..."},
     "capabilities":{}
   }}

<- {"jsonrpc":"2.0","id":1,"result":{
     "protocolVersion":"2025-11-25",
     "serverInfo":{"name":"sympho","version":"0.1.0"},
     "capabilities":{"tools":{"listChanged":false}},
     "instructions":"Resolve IDs before writes. Use expected_version..."
   }}

-> {"jsonrpc":"2.0","id":2,"method":"tools/list"}
```

2. Model-selected call

```
-> {"jsonrpc":"2.0","id":3,"method":"tools/call","params":{
     "name":"create_node",
     "arguments":{
       "module_id":"7ca8...",
       "title":"Structured concurrency and task groups",
       "description":"Understand child-task lifetime and cancellation.",
       "idempotency_key":"conversation-92:node-3"
     }
   }}
```

3. Structured result

```
<- {"jsonrpc":"2.0","id":3,"result":{
     "content":[{"type":"text","text":"Node created."}],
     "structuredContent":{
       "node":{"id":"b3d1...","title":"Structured concurrency...",
               "module_id":"7ca8...","version":1},
       "audit_id":"a91e...",
       "deduplicated":false
     },
     "isError":false
   }}
```

4. Correctable conflict

```
<- {"jsonrpc":"2.0","id":4,"result":{
     "content":[{"type":"text","text":"The node changed since version 3."}],
     "structuredContent":{
       "error":"conflict",
       "current_version":4,
       "recovery":"Call get_item, reconcile fields, then retry with version 4."
     },
     "isError":true
   }}
```

Appendix B. Sympho repository evidence

Finding	Location	Meaning
Native model container	Sympho/Sympho/SymphoApp.swift:15-40	Persistent SwiftData schema with 14 models.
Core hierarchy	Sympho/Sympho/Models/SymphoModels.swift:269-660	Domain, Track, Module, Project, Node, Resource.
Remote disabled	Sympho/Sympho/Data/SyncManager.swift:21	hasConfiguredRemoteBackend = false.
Simulated sync	Sympho/Sympho/Data/SyncManager.swift:25-56	Artificial delay and placeholder phases.
Placeholder pull	Sympho/Sympho/Data/SyncManager.swift:161-167	No remote fetch or merge implementation.
In-app AI	Sympho/Sympho/AI/AIService.swift:3-84	Typed node/module/project draft generation.
Sandbox entitlement	Sympho/Sympho/Sympho.entitlements:5-10	App sandbox, network client, app-scoped bookmarks.
Intended backend	Technical Architecture & Stack Selection 36fe5.md:7-20	Supabase/Postgres/Storage plus SwiftData offline cache.

Repository paths are relative to `/Users/mediaalamedia/sympho`. This analysis did not modify application code or database state.

References

[1] MCP architecture overview
https://modelcontextprotocol.io/docs/learn/architecture

[2] Understanding MCP servers
https://modelcontextprotocol.io/docs/learn/server-concepts

[3] MCP 2025-11-25 server/schema reference
https://modelcontextprotocol.io/specification/2025-11-25/schema

[4] MCP transports
https://modelcontextprotocol.io/specification/2025-06-18/basic/transports

[5] MCP security best practices
https://modelcontextprotocol.io/specification/2025-06-18/basic/security_best_practices

[6] MCP Inspector
https://modelcontextprotocol.io/docs/tools

[7] MCP client best practices
https://modelcontextprotocol.io/docs/develop/clients/client-best-practices

[8] Cursor MCP documentation
https://docs.cursor.com/context/model-context-protocol

[9] Cursor: Building an MCP Server
https://docs.cursor.com/en/guides/tutorials/building-mcp-server

[10] Claude Code MCP documentation
https://docs.anthropic.com/en/docs/claude-code/mcp

[11] OpenAI Codex manual: Model Context Protocol
https://developers.openai.com/codex/mcp

[12] Notion: Connecting to Notion MCP
https://developers.notion.com/docs/get-started-with-mcp

[13] Supabase: MCP Authentication
https://supabase.com/docs/guides/auth/oauth-server/mcp-authentication

[14] Supabase: OAuth 2.1 Server
https://supabase.com/docs/guides/auth/oauth-server

[15] Supabase: Deploy MCP servers
https://supabase.com/docs/guides/getting-started/byo-mcp

[16] OpenAI: Building MCP servers and security considerations
https://platform.openai.com/docs/mcp/

[17] MCP: Build with Agent Skills
https://modelcontextprotocol.io/tutorials/building-mcp-with-llms

[18] Notion MCP supported tools
https://developers.notion.com/guides/mcp/mcp-supported-tools

Source note

Standards and product capabilities change quickly. This report uses official MCP, OpenAI, Anthropic, Cursor, Notion, and Supabase documentation available on 19 June 2026. Validate protocol versions, SDK releases, authentication behavior, and client feature support again immediately before implementation.